



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e
Matematica Applicata (DIEM)

Documento di progetto - Gruppo 6

Software Architecture Design



Your favourite vibes

Versione 3

Membri

Autorino Luigi

Chirico Emanuel

Crisci Chiara

Graziuso Emanuela

1. Introduzione e panoramica

La presente relazione ha lo scopo di descrivere l'architettura e le funzionalità principali di '**VibeFlow**', un'applicazione desktop per la gestione e la riproduzione di tracce musicali. Il sistema mira a migliorare l'esperienza utente attraverso funzionalità avanzate di organizzazione, personalizzazione e controllo dinamico delle playlist e delle tracce.

I principali **obiettivi funzionali** del progetto sono:

- consentire la gestione completa delle tracce musicali: dalla riproduzione, alla visualizzazione del suo stato di avanzamento;
- permettere la creazione e modifica delle playlist;
- generare automaticamente delle playlist per decenni, generi e per caratteristiche (nuove uscite, esplicite e preferite)
- supportare differenti modalità di riproduzione (casuale, sequenziale e ciclica);
- garantire aggiornamenti in tempo reale dell'interfaccia;
- migliorare usabilità e flessibilità operativa.

Il documento è organizzato come segue: la Sezione 2 descrive i requisiti e come questi sono stati suddivisi in **User stories** dal team; la Sezione 3 descrive **l'architettura del sistema** e la scomposizione modulare in quattro domini (Track, Playlist, Playback Engine, View & Eventi); la Sezione 4 illustra la progettazione dell'interfaccia utente con wireframe e flussi di interazione.

VibeFlow è sviluppato in **Java 23.0.2** sfruttando **JavaFX** per l'interfaccia grafica. La scelta ricade su queste tecnologie per l'eccellente supporto ai design pattern utilizzati e per la capacità di gestire in modo nativo la scena grafica tramite thread dedicati alla riproduzione multimediale.

2. Requisiti e User Stories

Dall'analisi delle **epiche** sono state delineate le principali funzionalità alle quali il sistema deve assolvere e, una volta individuate le singole **user stories**, queste sono state divise in vari task dal team di sviluppo.

In particolare, il sistema deve consentire la **gestione completa del ciclo di vita delle tracce musicali**, includendo operazioni di **creazione**, **modifica** ed **eliminazione** (ID_1, ID_2, ID_3). Ogni traccia è caratterizzata da metadati obbligatori (titolo, autore, genere, durata, anno) e da tag visivi opzionali assegnabili dinamicamente (ID_21). Inoltre *VibeFlow* deve **supportare la creazione e gestione di playlist manuali** (ID_5, ID_6, ID_7, ID_8) e **playlist automatiche** generate algorithmicamente sulla base di metadati quali genere (ID_19), anno (ID_20), frequenza d'ascolto (ID_18) e tag visivi (ID_22).

Il motore di riproduzione deve supportare **tre modalità** distinte: sequenziale (ID_12), shuffle (ID_13) e loop (ID_14), con aggiornamento in tempo reale del player di riproduzione della traccia durante la riproduzione (ID_15). Le operazioni di aggiunta e rimozione tracce dalla libreria devono essere reversibili tramite meccanismo Undo (ID_16, ID_17).

Infine, le user stories ID_25, ID_26_ e ID_27 sono state pensate per soddisfare il requisito di rendere Il progetto funzionale e non solo a scopo dimostrativo.

ID	User Story	Priorità	Story Points
ID_1	Crea Track	High	3
ID_2	Modifica Track	High	1
ID_3	Rimuovi Track	High	1
ID_4	Visualizza Libreria Track	High	3
ID_5	Crea Playlist	High	3
ID_6	Aggiungi Track a Playlist	High	3
ID_7	Rimuovi Track da Playlist	High	1
ID_8	Elimina Playlist	High	1
ID_9	Riproduci Track e Playlist	High	5
ID_10	Sospendi e Riprendi Riproduzione	High	5

ID_11	Skip Track	High	5
ID_12	Riproduzione 'Sequenziale'	High	3
ID_13	Riproduzione 'Shuffle'	High	3
ID_14	Riproduzione 'Loop'	High	3
ID_15	Aggiornamento Real-Time Player	High	5
ID_16	Undo Aggiunta Traccia	Medium	2
ID_17	Undo Rimozione Traccia	Medium	2
ID_18	Playlist MostPlayed	Medium	3
ID_19	Playlist Genre	Medium	3
ID_20	Playlist Year	Medium	3
ID_21	Assegna Tag Visivi	Medium	2
ID_22	Playlist Tag Visivi	Medium	3
ID_23	Riordina Brani Playlist	Low	3
ID_24	Rinomina Playlist	Low	1
ID_25	Progettazione GUI	Medium	5
ID_26	Creazione tracce '.mp3'	Medium	3
ID_27	Riproduzione tracce '.mp3'	Medium	3

Il lavoro di sviluppo è stato pianificato in **tre sprint** seguendo la metodologia Agile Scrum. Le user stories sono state priorizzate secondo il modello MoSCoW (High/Medium/Low) e stimate in Story Points da un valore minimo di 1 ad un valore massimo di 8. Il totale degli Story Points per l'intero progetto ammonta a 78 Story Points.

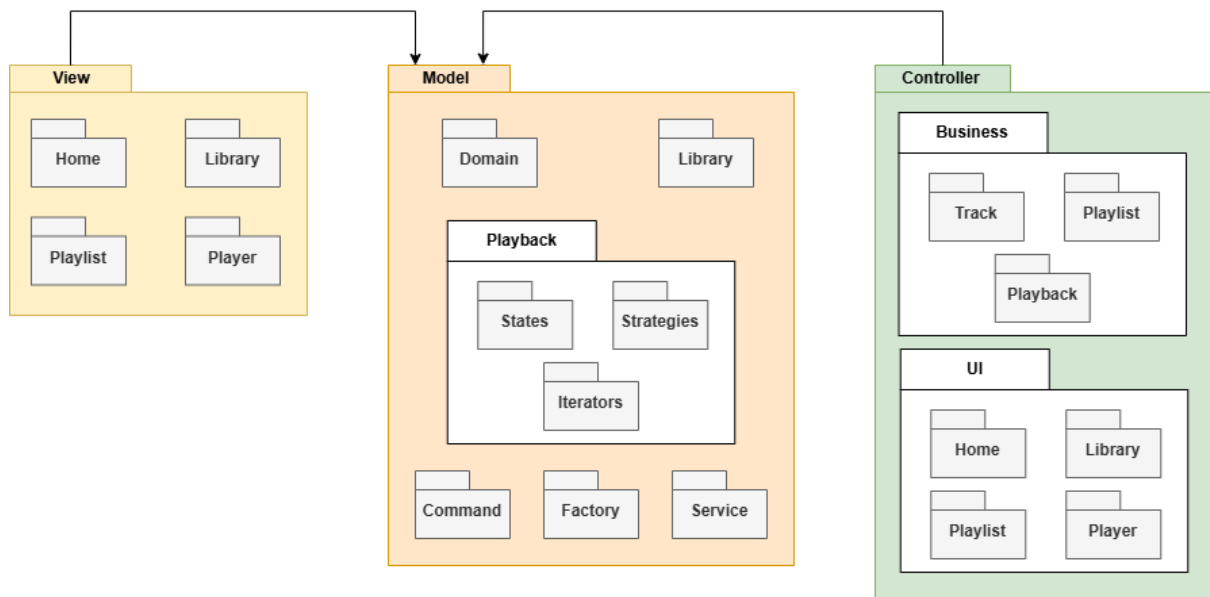
3. Architettura del Sistema

La presente sezione descrive l'architettura complessiva di VibeFlow, definendo i principali componenti del sistema e le scelte progettuali di alto livello adottate per tradurre i requisiti funzionali in una struttura software solida, modulare e manutenibile.

L'architettura del sistema **VibeFlow** si basa sul pattern architetturale *Model-View-Controller*. Al fine di garantire un'elevata coesione e un basso accoppiamento, il sistema adotta una rigorosa ripartizione dei compiti attraverso tre layer principali. Tale impostazione rispetta il **principio dell'inversione delle dipendenze**, in base al quale la logica di presentazione e di controllo dipendono dal modello dei dati, mentre quest'ultimo opera in totale isolamento. Inoltre, l'applicazione del MVC consente di migliorare la scalabilità del sistema, facilitare la manutenzione e rendere più agevole l'evoluzione delle singole componenti software.

- ❖ Il layer **Model** rappresenta il nucleo applicativo del sistema. Esso gestisce in via esclusiva la logica di business, l'accesso ai dati e lo stato dell'applicazione. Al suo interno è suddiviso in sei sotto-pacchetti funzionali. Il pacchetto *Domain* contiene le entità fondamentali del dominio applicativo. Il pacchetto *Library* gestisce i cataloghi di brani e playlist, integrando le interfacce del pattern Observer per una notifica degli eventi disaccoppiata. Il pacchetto *Playback* ospita il motore di riproduzione centrale: al suo interno, l'architettura isola i comportamenti complessi nei sub-package States per l'implementazione del pattern State, Strategies per le logiche di rimescolamento e ripetizione tramite il pattern Strategy, e Iterators per l'attraversamento delle code di riproduzione. Il pacchetto *Command* implementa l'omonimo pattern, incapsulando le azioni dell'utente in oggetti concreti per garantire la reversibilità delle operazioni. Il pacchetto *Factory* utilizza il pattern Factory Method per astrarre e automatizzare la creazione delle raccolte. Infine, il pacchetto *Service* espone i metodi principali per il controllo della riproduzione.
- ❖ Il layer **Controller** agisce da intermediario tra l'interfaccia utente e il modello. Per rispettare il *Single Responsibility Principle* all'interno del framework JavaFX, questo livello è stato strutturalmente scomposto in due macro-aree. L'area *Business* contiene i controller logici, i quali sono completamente indipendenti dall'interfaccia grafica: il loro ruolo è ricevere gli input, applicare le regole di dominio e orchestrare il Model delegando le modifiche di stato. L'area *UI* contiene, invece, i controller che gestiscono l'interfaccia grafica. Il loro compito esclusivo è intercettare gli eventi generati dall'utente per inoltrarli ai controller di business e, contemporaneamente, agire da osservatori nei confronti del Model per aggiornare in tempo reale gli elementi visivi.

- ❖ Il layer **View** è dedicato unicamente alla presentazione passiva. Seguendo il *principio Open-Closed*, questo livello non contiene logica applicativa né file sorgente Java, ma è preposto all'organizzazione strutturale dei file di markup e dei fogli di stile. La sua organizzazione interna in sotto-pacchetti riflette in modo simmetrico la struttura dell'area UI del Controller. Questa scelta progettuale assicura una completa flessibilità nella sostituzione o nell'evoluzione del comparto grafico senza comportare alcun impatto sull'infrastruttura logica del sistema.



Per mostrare nel concreto l'implementazione, le classi del sistema VibeFlow sono state suddivise in diagrammi tematici: si parte dal **modello di dominio**, che definisce le entità fondamentali del sistema, per poi analizzare ciascun **design pattern GoF** applicato (Command, Observer, Factory Method, State e Strategy).

3.1 Il Modello di dominio

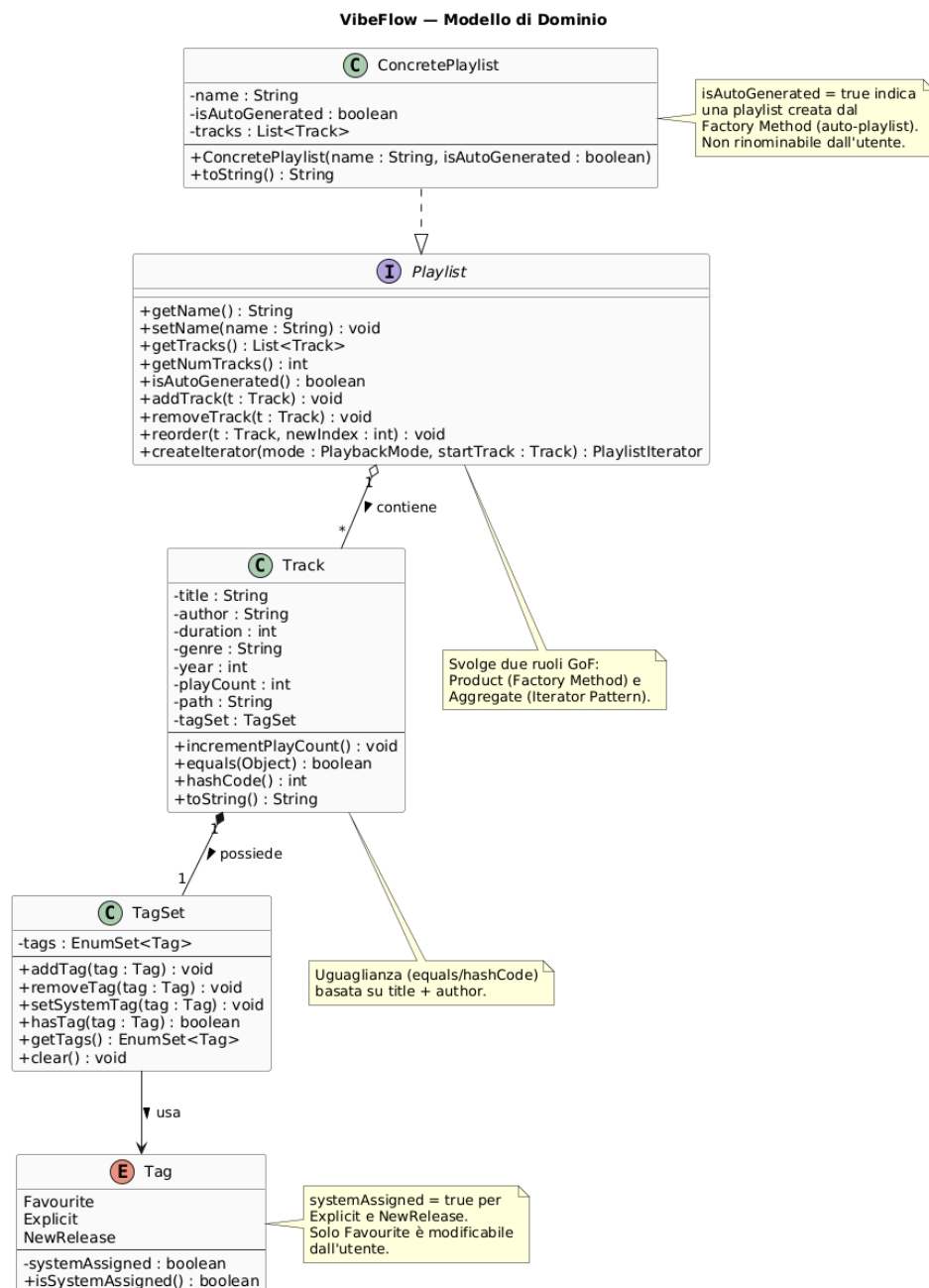
Questo modulo descrive le entità fondamentali che costituiscono il nucleo informativo di VibeFlow e le scelte architetturali adottate per la loro istanziazione. L'obiettivo primario in questa fase di progettazione è stato garantire una netta separazione tra la rappresentazione dei dati e le logiche di creazione complessa, favorendo la flessibilità e l'estensibilità del codice.

Il dominio applicativo ruota attorno a tre concetti cardine, modellati attraverso un approccio Object-Oriented rigoroso:

- **Track:** Rappresenta la singola traccia musicale. Incapsula i metadati fondamentali (titolo, autore, genere, durata, anno, numero di riproduzioni e percorso del file '.mp3'). L'uguaglianza logica tra due istanze di Track è stata

definita basandosi sulla combinazione di titolo e autore, prevenendo così la duplicazione accidentale dei brani all'interno delle collezioni.

- **Tag e TagSet:** Per arricchire la classificazione dei brani, è stato introdotto il concetto di "Tag visivo". L'enumerazione **Tag** definisce le etichette di sistema (**Favourite**, **Explicit**, **NewRelease**). La classe **TagSet** agisce come contenitore e gestisce la logica di assegnazione, distinguendo tra tag modificabili dall'utente e tag assegnati in sola lettura dal sistema.
- **Playlist (Interfaccia) e ConcretePlaylist:** Al fine di disaccoppiare l'uso delle playlist dalla loro specifica implementazione, **Playlist** è stata definita come un'interfaccia. Essa dichiara i contratti per le operazioni standard (aggiunta, rimozione, ordinamento e ottenimento delle tracce). La sua realizzazione concreta, **ConcretePlaylist**, introduce inoltre il flag *isAutoGenerated*, fondamentale per inibire modifiche manuali sulle playlist generate algoritmicamente dal sistema.



3.2 Generazione algoritmica e Pattern Factory Method

Una delle funzionalità core di VibeFlow richiede la generazione automatica di playlist basate su specifici criteri di filtraggio (per genere, anno, tag o frequenza di ascolto). Per risolvere il problema di istanziare queste collezioni senza accoppiare strettamente i Controller di business alle logiche di filtraggio, è stato applicato il **Design Pattern GoF: Factory Method**. Questo pattern creazionale permette di delegare la responsabilità della creazione degli oggetti a sottoclassi specializzate.

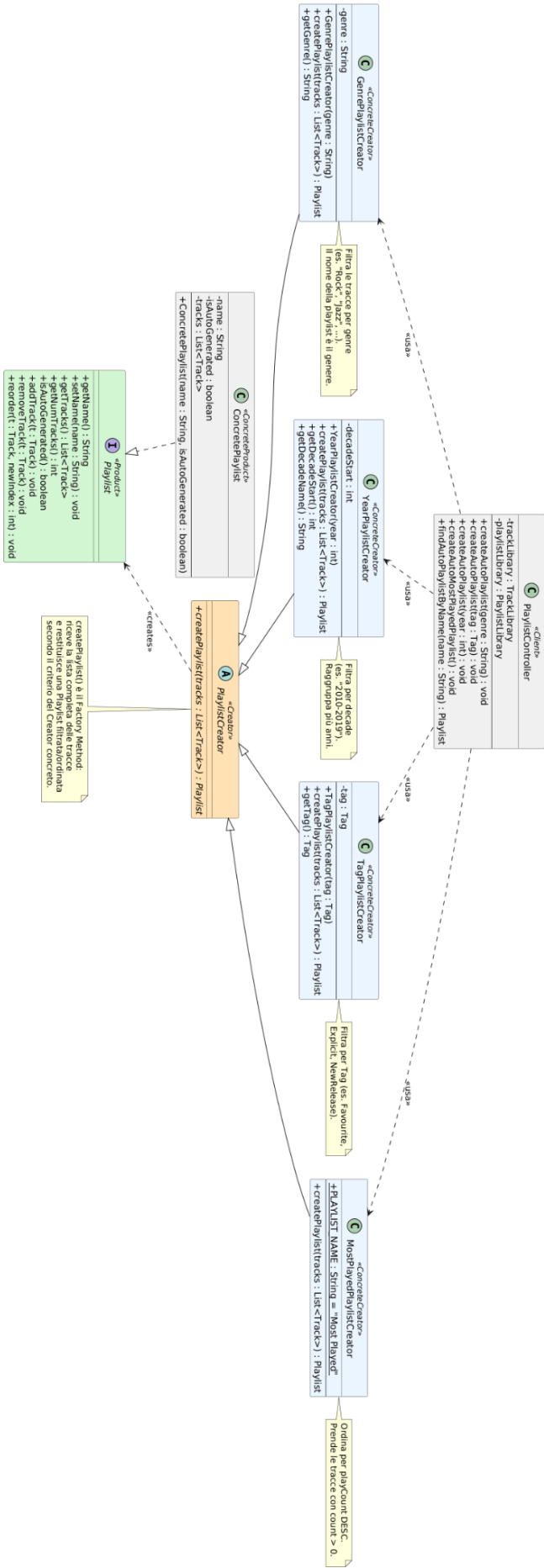
La mappatura del pattern all'interno dell'architettura di VibeFlow è la seguente:

- **Product:** L'interfaccia `Playlist`, che rappresenta il prodotto astratto restituito dal factory.
- **Concrete Product:** `ConcretePlaylist`, ovvero l'oggetto reale istanziato, opportunamente contrassegnato come auto-generato.
- **Creator:** La classe astratta `PlaylistCreator`, la quale dichiara il factory method `createPlaylist()`.
- **Concrete Creator:** Le classi specializzate come `GenrePlaylistCreator`, `YearPlaylistCreator`, `TagPlaylistCreator` e `MostPlayedPlaylistCreator`.

Vale la pena sottolineare che tutti e quattro i `ConcreteCreator` istanziano lo stesso `ConcretePlaylist`, ma con parametri e logiche di filtraggio differenti. La variabilità del pattern risiede quindi nel *comportamento* del creator, non nella struttura del Product: infatti ogni creator filtra la libreria secondo il proprio criterio e produce una `ConcretePlaylist` marcata con `isAutoGenerated = true`, flag che ne impedisce la rinomina e l'eliminazione manuale da parte dell'utente.

L'applicazione del Factory Method garantisce il pieno rispetto del **Principio Open-Closed (OCP)**: qualora in futuro emergesse il requisito di aggiungere un nuovo criterio di generazione automatica, sarà sufficiente introdurre una nuova classe che estende `PlaylistCreator`, senza dover modificare o alterare in alcun modo il codice preesistente del Controller o degli altri Creator.

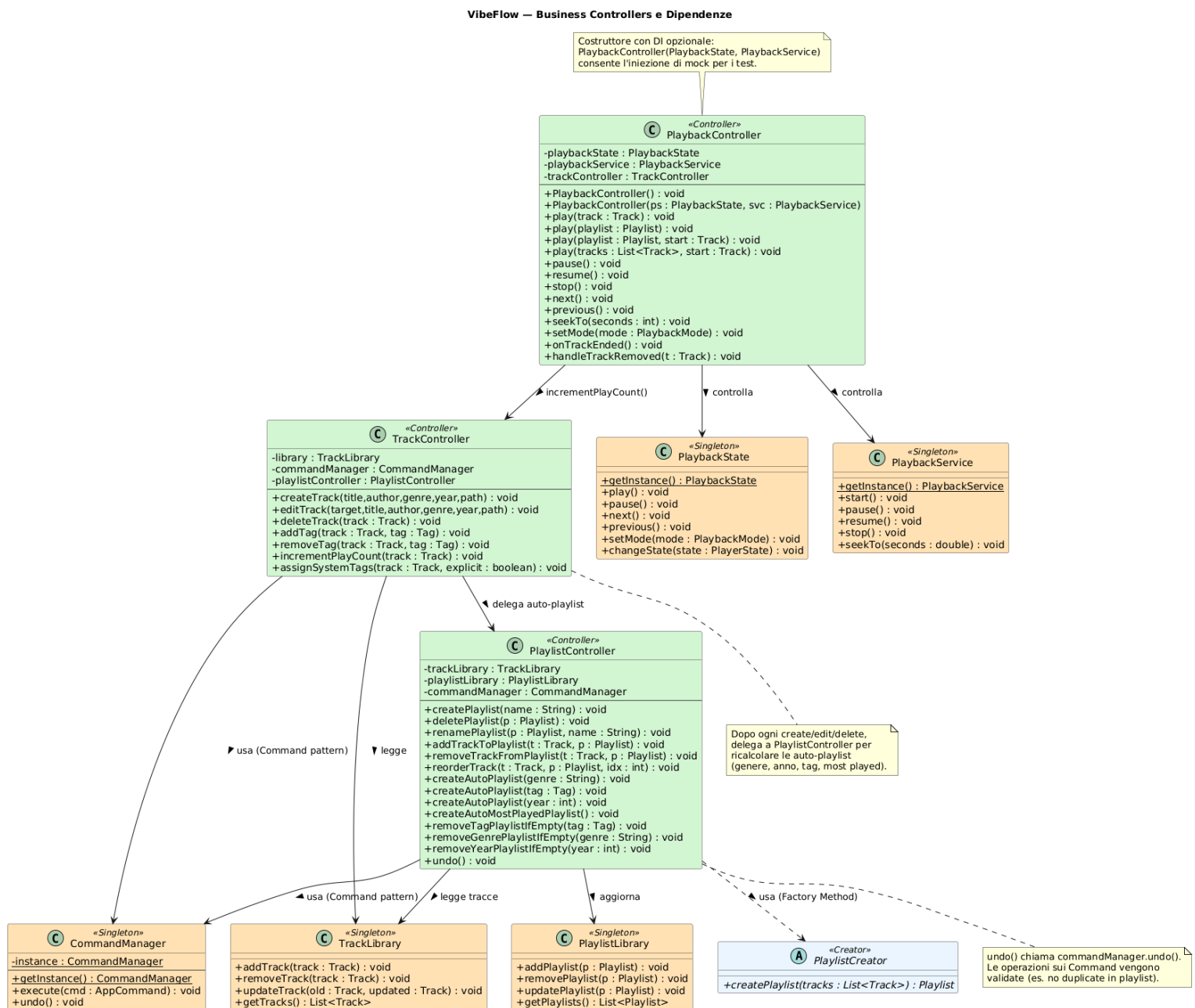
VibeFlow – Pattern Factory Method (Playlist Automatiche)



3.3 I Business Controller e il Pattern Command

All'interno dell'architettura MVC di VibeFlow, il layer dei Controller è stato strutturalmente suddiviso per separare la gestione delle evoluzioni dell'interfaccia dalle regole core dell'applicazione. Questa sezione si concentra sui **Business Controller** e su come il sistema garantisca l'esecuzione sicura, centralizzata e reversibile delle operazioni di modifica sulla libreria musicale e sulle playlist.

I controller logici principali, **TrackController** e **PlaylistController**, operano a livello di business e sono completamente agnostici rispetto al framework grafico JavaFX. Il loro compito fondamentale è ricevere le richieste di manipolazione dei dati, validarle e coordinare le modifiche al modello.



Tuttavia, l'esecuzione diretta di operazioni complesse (come l'eliminazione di una traccia, che richiede la rimozione simultanea della stessa da tutte le playlist in cui compare) avrebbe introdotto un forte accoppiamento e reso complessa l'implementazione delle funzionalità di annullamento. Per superare questo limite architetturale è stato applicato il **Design Pattern GoF: Command**, che trasforma ogni richiesta di modifica in un oggetto autonomo, contenente tutte le informazioni necessarie per la sua esecuzione. Questa astrazione permette di parametrizzare i client rispetto alle azioni, accodare le richieste e, soprattutto, supportare le operazioni di annullamento (Undo).

La mappatura del pattern all'interno di VibeFlow si articola nei seguenti ruoli:

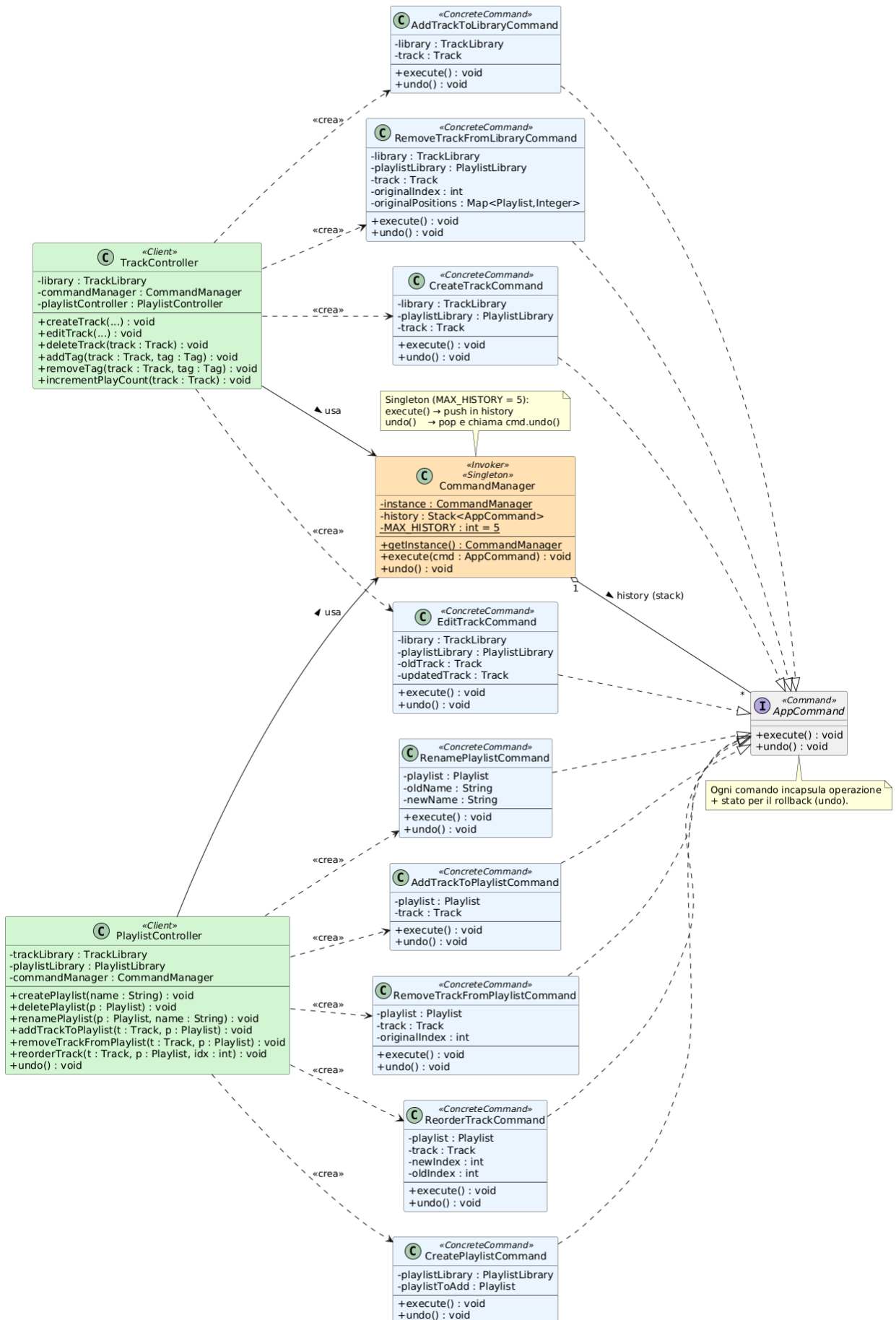
- **Command (Interfaccia):** Corrisponde a `AppCommand`, un'interfaccia pura che definisce il contratto per tutti i comandi annullabili del sistema attraverso due metodi fondamentali: `execute()` per l'applicazione dell'azione e `undo()` per il ripristino dello stato precedente.
- **Concrete Command:** le classi specializzate che incapsulano un'azione atomica e lo stato necessario al suo eventuale rollback. Troviamo comandi legati alla gestione delle tracce e comandi legati alle playlist.
- **Invoker:** Il `CommandManager`, implementato come un **Singleton centralizzato**. È responsabile di ricevere i comandi, eseguirli e mantenerli in uno stack LIFO per supportare l'undo. La natura Singleton garantisce che qualsiasi operazione, indipendentemente dal controller che la origina, venga registrata nella stessa pila, rendendo l'undo globalmente coerente.
- **Receiver:** Le librerie core del dominio (`TrackLibrary` e `PlaylistLibrary`), che contengono l'effettiva logica algoritmica e le strutture dati su cui operare.
- **Client:** I controller di business (`TrackController` e `PlaylistController`), responsabili dell'intercettazione dell'intento dell'utente e della conseguente istanziazione del `ConcreteCommand` appropriato.

Il punto di forza di questa implementazione risiede nella capacità di ogni comando concreto di catturare lo *stato del sistema* prima che l'operazione venga eseguita, garantendo un rollback sicuro e privo di effetti collaterali.

Quando il Controller di business consegna il comando al `CommandManager`, quest'ultimo invoca il metodo `execute()`. Il manager esegue l'azione e, se l'operazione va a buon fine, inserisce l'oggetto comando all'interno di una struttura dati di tipo `Stack<AppCommand>`. Al fine di ottimizzare l'uso della memoria ed evitare overhead prestazionali, la history è limitata superiormente ad una dimensione massima stabilita (`MAX_HISTORY = 5`).

Quando l'utente richiede un annullamento, il `CommandManager` effettua un'operazione di `pop()` dallo stack, estrae l'ultimo comando eseguito e ne invoca il metodo `undo()`, il quale ripristina lo stato del dominio attingendo alle informazioni precedentemente congelate al suo interno.

VibeFlow — Pattern Command



L'introduzione del pattern Command offre una serie di vantaggi cruciali per la manutenibilità del software:

- **Alta Coesione e Basso Accoppiamento:** I controller non contengono script operativi complessi né logiche di rollback; si limitano a istanziare oggetti comando, delegando l'infrastruttura transazionale al manager.
- **Estendibilità (Principio Open-Closed):** L'aggiunta di nuove operazioni reversibili richiede unicamente l'aggiunta di una nuova classe concreta che implementi `AppCommand`, senza la necessità di modificare la logica strutturale del `CommandManager`.

3.4 Motore di Riproduzione Audio: Pattern Strategy, Iterator e State

Il nucleo operativo di VibeFlow, deputato alla gestione del flusso audio, è incapsulato nel livello di riproduzione coordinato dal `PlaybackController`. Questo modulo affronta due complessità architetturali distinte: la gestione in tempo reale dello stato del player multimediale e la risoluzione algoritmica della navigazione tra i brani. Al fine di garantire un'elevata coesione e prevenire la proliferazione di istruzioni condizionali complesse, il design si avvale della collaborazione sinergica di tre pattern comportamentali: **State**, **Strategy** e **Iterator**.

C'è tuttavia da fare una premessa doverosa sul ruolo dell'`Iterator`: in VibeFlow esso non segue il caso d'uso classico del pattern, ossia la scansione sequenziale esaustiva di una collezione all'interno di un ciclo, bensì una variante **event-driven**. L'iteratore avanza di un solo passo per volta, su richiesta esplicita dell'utente o al termine naturale di una traccia. Questo modello di navigazione si adatta perfettamente al dominio di un media player, in cui la coda di riproduzione può essere interrotta, ripresa o invertita in qualsiasi momento.

La struttura GoF rimane intatta: `PlaylistIterator` è l'interfaccia `Iterator` con `hasNext()`, `next()`, `hasPrevious()` e `previous()`; i tre `ConcreteIterator`, `SequentialIterator`, `ShuffleIterator`, `LoopIterator`, incapsulano ciascuno un algoritmo di navigazione distinto; `ConcretePlaylist` funge da `Collection`, delegando la creazione dell'iteratore appropriato alla Strategy attiva tramite `createIterator()`.

Un ulteriore requisito di dominio critico impone al sistema di supportare diverse modalità di riproduzione: sequenziale, shuffle e loop. Per evitare che il codice di navigazione sia vincolato a una logica fissa, la variabilità degli algoritmi è gestita dal Pattern **Strategy** in stretta sinergia con l'iteratore precedentemente introdotto. In particolare, per questo meccanismo si riconoscono tre ruoli fondamentali:

- **Il Ruolo della Strategy:** L'interfaccia **PlaybackMode** agisce come Strategy, e le sue declinazioni concrete (**SequentialMode**, **ShuffleMode**, **LoopMode**) definiscono l'*algoritmo* con cui la collezione deve essere attraversata. Questa interfaccia espone un metodo **getIterator()** che si comporta a tutti gli effetti come un Factory Method, delegando alle strategie concrete l'istanziamento dell'iteratore appropriato.
- **Il Ruolo dell'Iterator:** l'interfaccia **PlaylistIterator** rispetta il contratto GoF con **hasNext()** e **next()**, estendendolo con **hasPrevious()** e **previous()** per supportare la navigazione bidirezionale tipica di un media player, e con **updateTracks()** per aggiornare la coda quando la libreria viene modificata durante la riproduzione.
- **Il Ruolo della Collection:** l'interfaccia **Playlist** funge da collezione e delega la creazione dell'iteratore alla Strategy attiva in quel momento nel sistema tramite il metodo **createIterator()**.

Le implementazioni concrete dell'Iterator risolvono le complessità di navigazione in totale autonomia:

- **SequentialMode & SequentialIterator:** Gestiscono il normale attraversamento lineare della lista basato su indice, con arresto logico al raggiungimento dell'ultimo brano.
- **ShuffleMode & ShuffleIterator:** Implementano la riproduzione casuale. Per supportare l'operazione *previous()* in modo coerente, l'iteratore mantiene due liste interne: *remaining* (brani non ancora riprodotti) e *history* (sequenza casuale già percorsa).
- **LoopMode & LoopIterator:** Permettono la navigazione circolare infinita della playlist. In questo scenario, i metodi *hasNext()* e *hasPrevious()* restituiscono costantemente true, calcolando l'avanzamento del cursore tramite un'operazione di modulo rispetto alla dimensione della collezione.

L'integrazione dei pattern **Strategy** e **Iterator** permette di astrarre completamente le logiche di indicizzazione e rimescolamento dal Context e dal client. Questi ultimi interagiscono esclusivamente tramite interfacce standard, invocando metodi quali *iterator.next()*, approccio che favorisce la flessibilità del sistema e il rispetto del

Single Responsibility Principle.

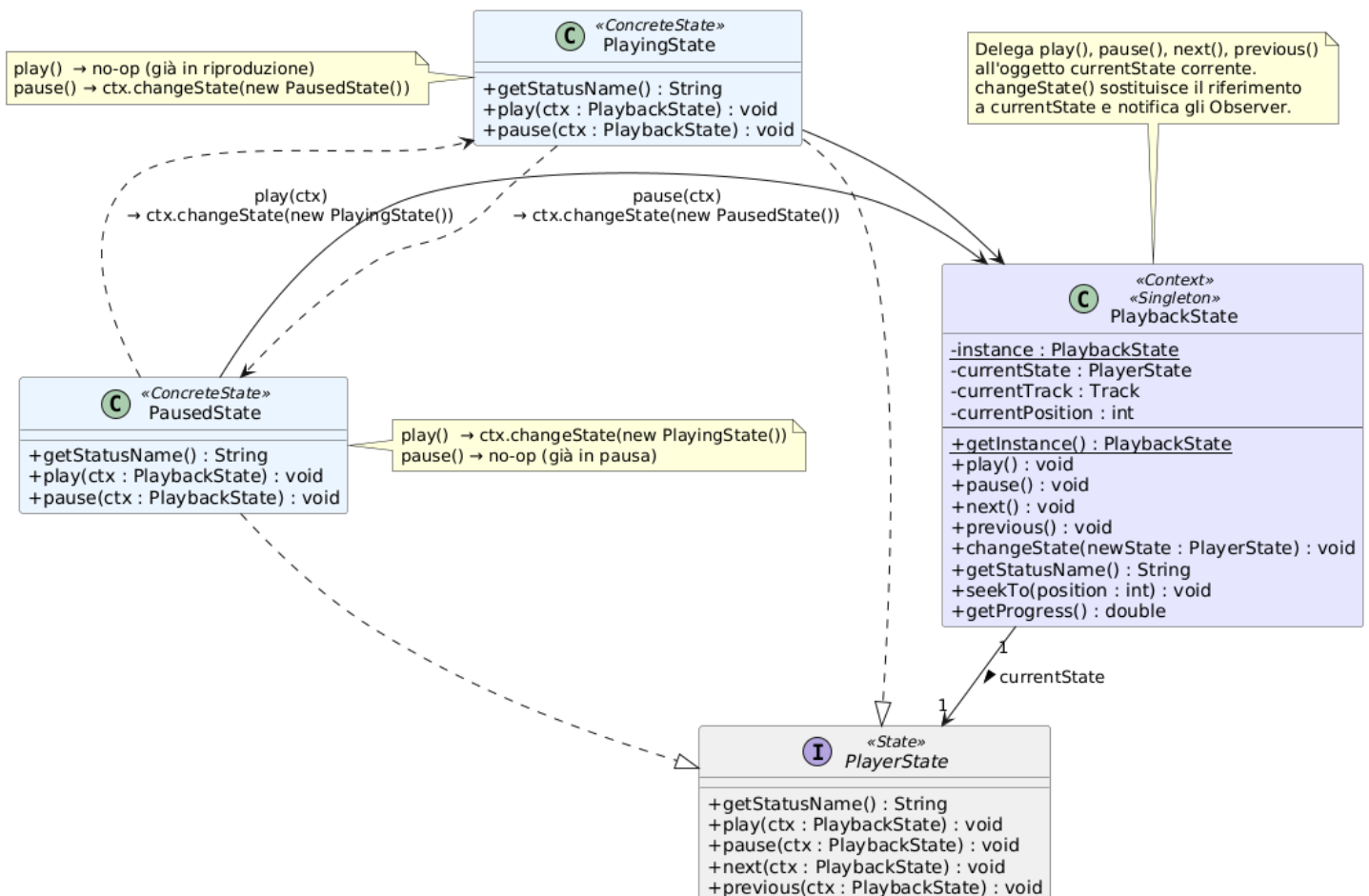
Tale sinergia architetturale assicura una netta **Separation of Concerns**: mentre la playlist si occupa della mera gestione dei dati in memoria, le responsabilità relative alla navigazione e all'istanziamento dei componenti rimangono isolate in moduli dedicati.

Infine, per gestire la variabilità delle risposte del motore audio agli input utente è stato adottato il **Design Pattern GoF State**. Questa scelta evita l'impiego di approcci procedurali basati su strutture condizionali, delegando il comportamento specifico agli oggetti di stato. Il motore audio deve reagire in modo diverso agli input dell'utente a seconda dello stato in cui si trova l'applicazione.

La modellazione del pattern prevede:

- **Context:** La classe `PlaybackState`, definita come **Singleton**, che funge da contesto globale. Essa mantiene i riferimenti alla *currentTrack*, alla *currentPosition* e a un'istanza dell'interfaccia `PlayerState` che ne rappresenta lo stato logico corrente.
- **State (Interfaccia):** L'interfaccia `PlayerState` dichiara i metodi transazionali come *play()*, *pause()*, *next()* e *previous()*.
- **Concrete State:** Le implementazioni concrete `PlayingState` e `PausedState` incapsulano il comportamento specifico.

VibeFlow — Pattern State (Riproduzione Audio)



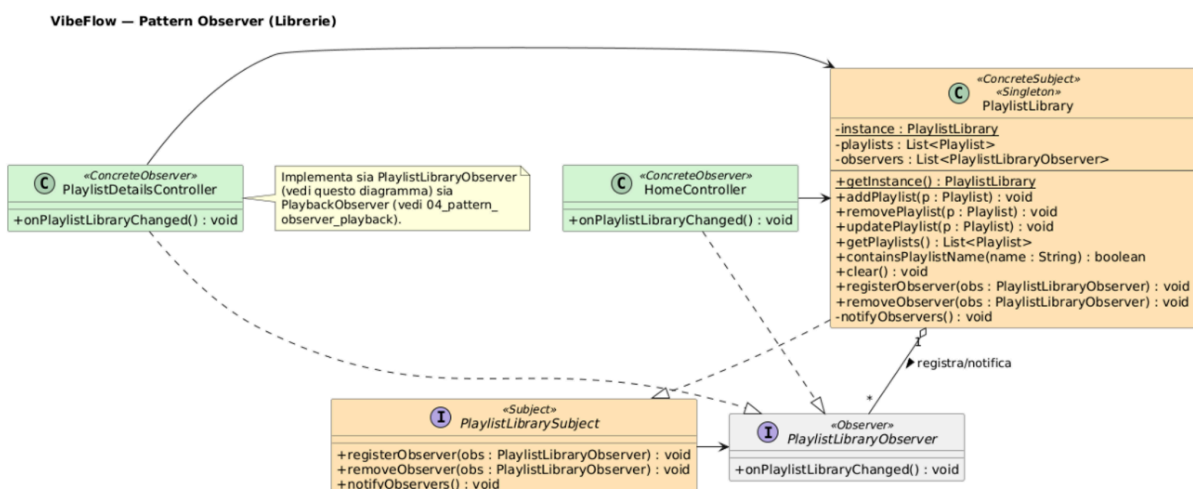
3.5 Architettura Reattiva e Sincronizzazione dell'Interfaccia: Il Pattern Observer

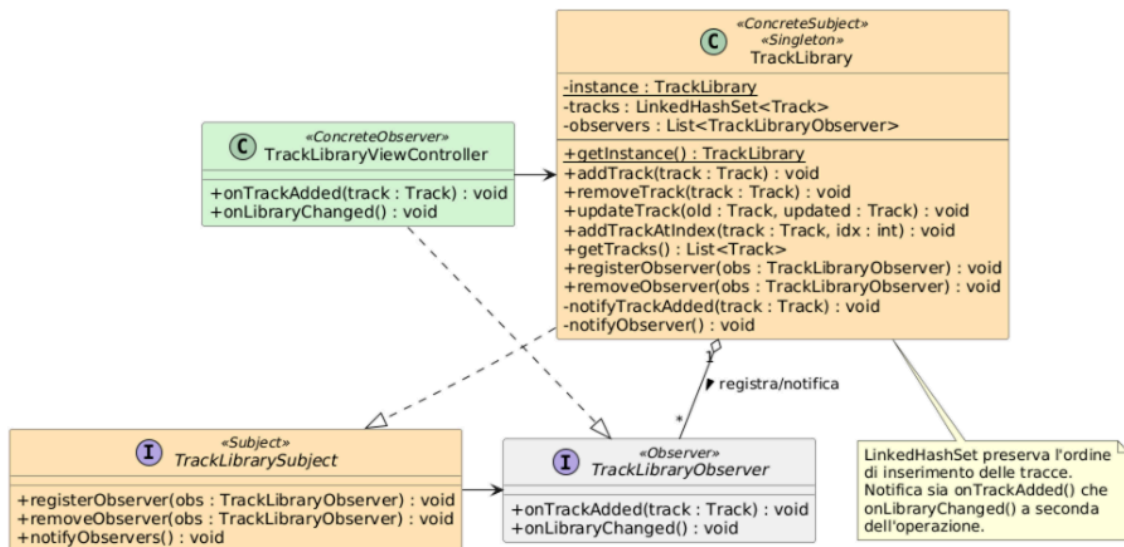
Per preservare l'integrità del pattern architetturale MVC e rispettare il principio di Inversione delle Dipendenze, il layer del Modello in VibeFlow deve operare in totale isolamento, senza possedere alcun riferimento diretto alle classi del framework grafico JavaFX. Tuttavia, sorge la necessità critica di mantenere l'interfaccia utente (*View*) costantemente sincronizzata con le mutazioni dello stato interno.

Per risolvere questo problema di comunicazione asincrona e unidirezionale, l'architettura si affida in modo estensivo al **Design Pattern GoF: Observer**, declinato in due contesti paralleli ma funzionalmente distinti: l'osservazione delle librerie e l'osservazione del motore di riproduzione.

Il primo contesto applicativo riguarda la mutazione delle collezioni di dati. Le entità centrali *TrackLibrary* e *PlaylistLibrary*, implementate come **Singleton**, assumono il ruolo di *Concrete Subject*. Esse estendono concettualmente le interfacce *TrackLibrarySubject* e *PlaylistLibrarySubject*, esponendo i metodi classici per la registrazione e la deregistrazione degli *Observer* (*registerObserver()*, *removeObserver()*).

I controller dedicati alla presentazione visiva, come *TrackLibraryViewController*, *HomeController* e *PlaylistDetailsController*, fungono da *Concrete Observer*. Implementando le interfacce *TrackLibraryObserver* e *PlaylistLibraryObserver*, essi si iscrivono in ascolto delle librerie durante la fase di inizializzazione. Quando l'utente esegue un'operazione di modifica, la *TrackLibrary* invoca il metodo *notifyObserver()*. Questo scatena una notifica broadcast, informando le viste dell'avvenuto cambiamento e innescando il ricalcolo e il ridisegno delle tabelle grafiche, senza che la libreria sappia *chi* o *come* stia visualizzando i dati.





Il secondo contesto applicativo, caratterizzato da un'altissima frequenza di aggiornamento, riguarda il player multimediale. In questo scenario, la classe **PlaybackState** ossia il subject che notifica gli observer ad ogni variazione, è caratterizzato da una densità semantica su cui è opportuno porre attenzione:

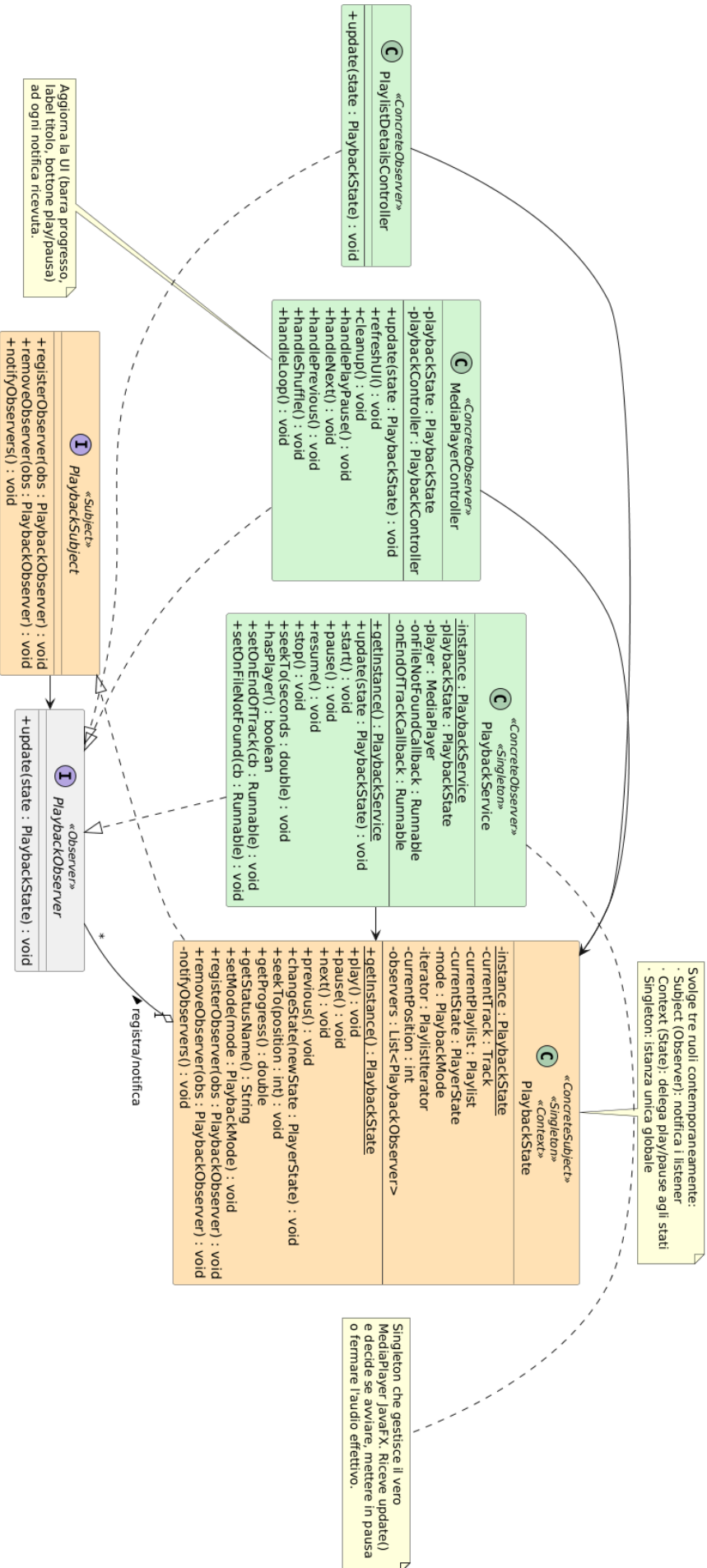
1. **Singleton:** garantendo un unico punto di accesso globale allo stato di riproduzione.
2. **Context (Pattern State):** in quanto gestisce lo stato **logico** del player, notificando gli observers quando avviene un cambiamento notevole (play, pause, next, previous)
3. **Subject (Pattern Observer):** Gestisce la lista dei **PlaybackObserver** interessati all'avanzamento audio.

I controller della UI, in particolare il **MediaPlayerController** e il **PlaylistDetailsController**, implementano l'interfaccia **PlaybackObserver**. Ad ogni tick del motore audio o ad ogni cambio di stato transazionale, il **PlaybackState** notifica questi osservatori invocando il metodo *update()*.

Il **MediaPlayerController** riceve l'aggiornamento e utilizza le informazioni fornite dal Subject per aggiornare in tempo reale la GUI: muove la barra di progresso, aggiorna la label del titolo e del tempo rimanente, commuta l'icona del bottone play/pausa.

L'implementazione del pattern Observer ha consentito di raggiungere un **disaccoppiamento** quasi totale tra logica e presentazione. L'aggiunta di una nuova vista richiederà semplicemente la creazione di una nuova classe che implementa le interfacce Observer preesistenti, senza alcuna necessità di alterare le classi del *Model*. Inoltre, l'utilizzo di interfacce Observer distinte aderisce al **Principio di Segregazione delle Interfacce**, garantendo che ogni controller venga notificato esclusivamente per gli eventi di suo reale interesse.

VibeFlow — Pattern Observer (Playback)



4. Progettazione dell'interfaccia utente

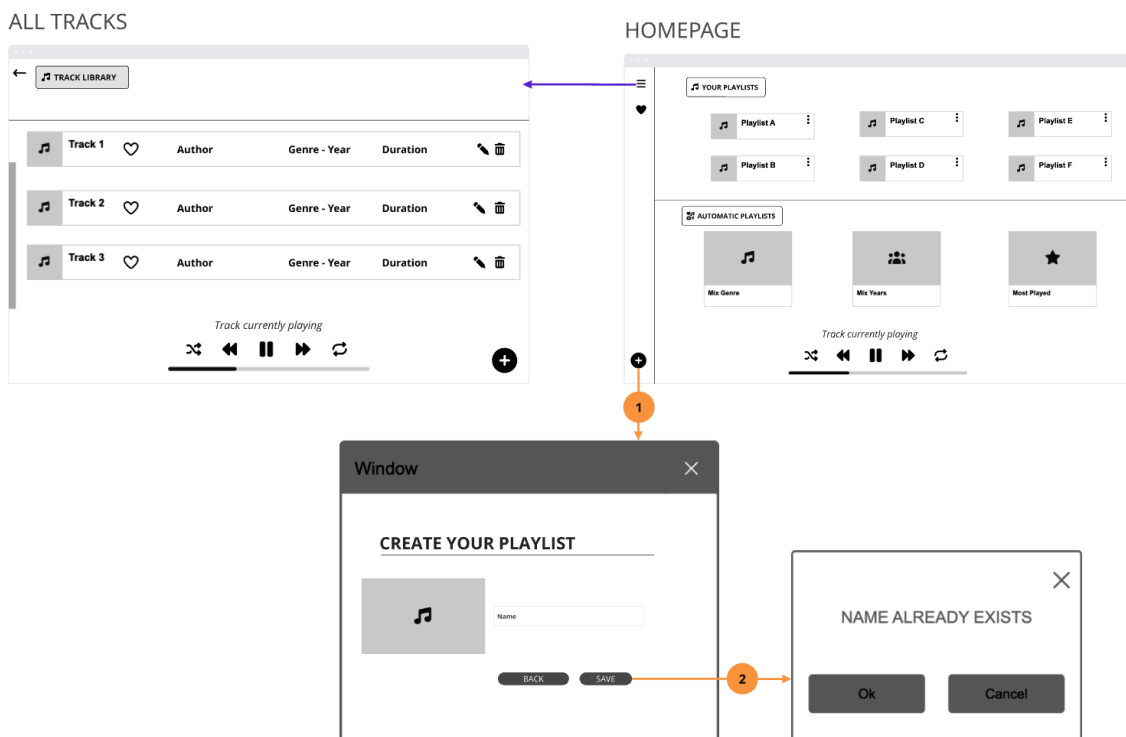
La fase di prototipazione, condotta tramite la realizzazione di **Wireframe** su piattaforma Miro, è risultata propedeutica allo sviluppo dell'interfaccia grafica definitiva. Tale attività ha avuto come obiettivo primario la definizione strutturale delle interfacce e l'analisi dei flussi di navigazione, coprendo gli scenari operativi relativi alla gestione delle playlist, alla navigazione nella libreria musicale, alle funzioni di creazione e modifica dei brani, nonché al controllo della riproduzione audio.

Tali prototipi hanno fatto da riferimento concettuale e strutturale, delineando le basi per la successiva implementazione tecnica.

4.1 Descrizione flussi d'interazione

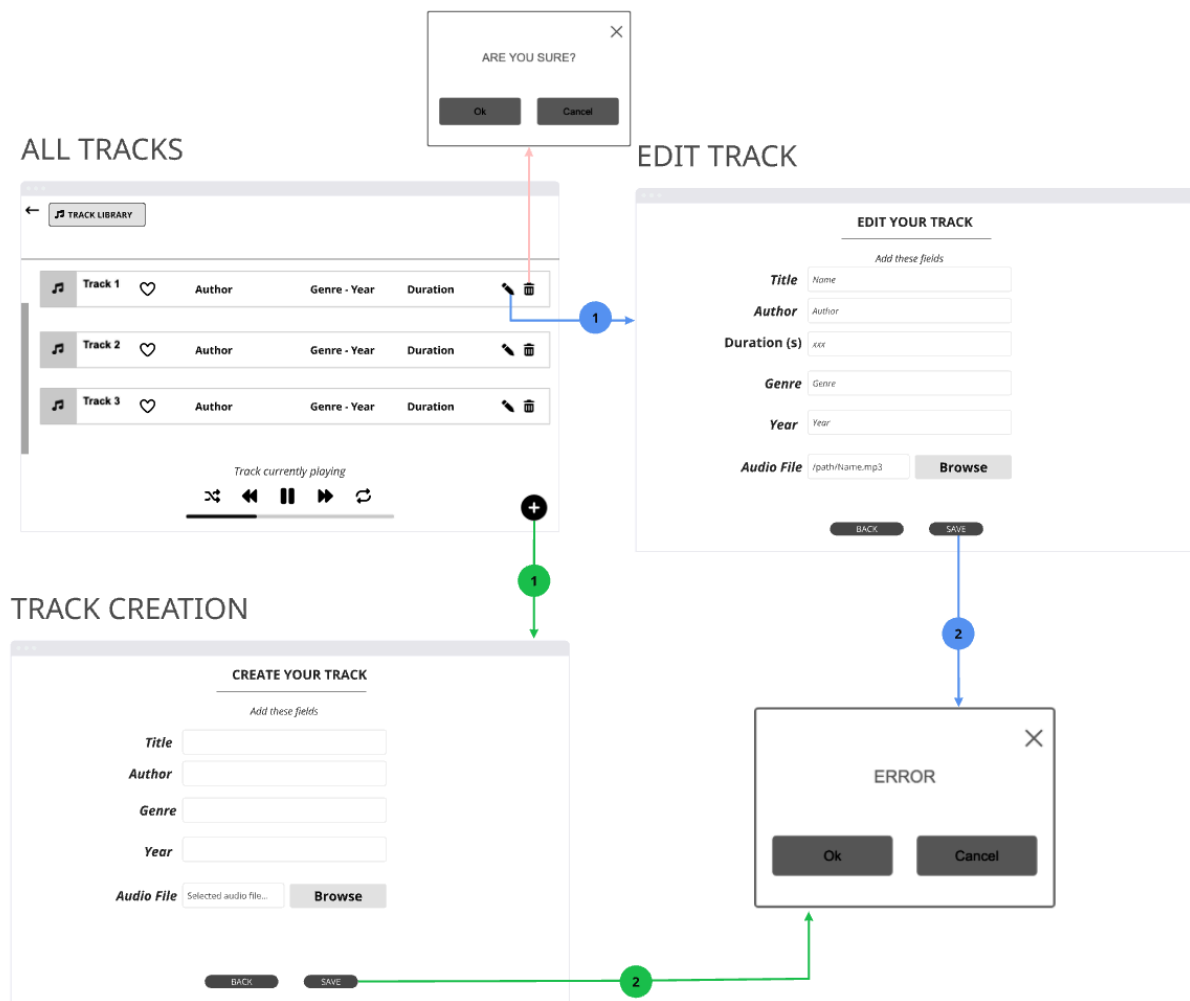
La presente sezione descrive i principali **flussi di interazione** tra le schermate dell'applicazione, illustrando come l'utente navighi da una vista all'altra in risposta alle proprie azioni.

Flusso 1 – HomePage, Playlist Creation, All Tracks



Il primo flusso illustra due percorsi distinti. Il primo mostra come, partendo dalla **Homepage**, sia possibile aprire il dialog di **Playlist Creation** interagendo col pulsante '+'; il tentativo di creare una playlist utilizzando un nome già esistente genera un dialog di errore. Il secondo flusso mostra come, a partire dalla schermata **Homepage**, sia possibile accedere alla schermata **All Tracks** tramite l'icona '≡'.

Flusso 2: Track Creation, Track Editing, Track Deletion



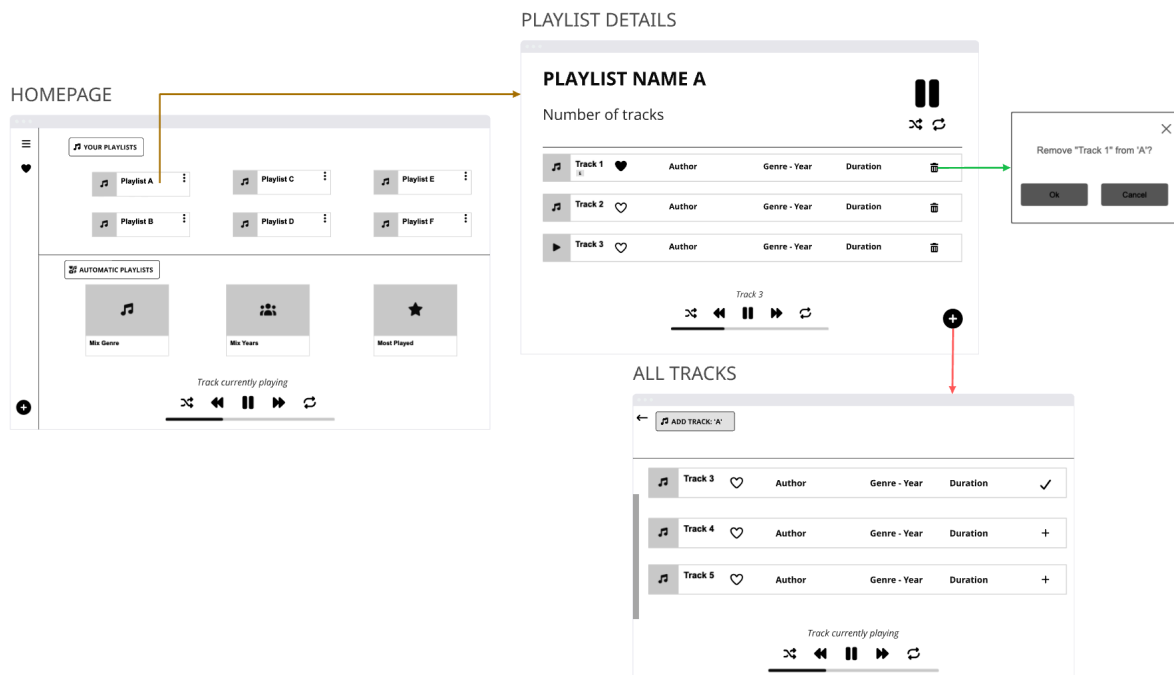
Il secondo flusso mostra tutte le operazioni di gestione delle tracce consentite nella schermata **All Tracks**.

Interagendo con l'icona '✎' su un brano si accede alla schermata **Edit Track**, precompilata con i metadati correnti. Da qui, è possibile aggiornare i dati della traccia coerentemente ai vincoli di business stabiliti. Qualora la modifica non rispettasse tali regole, il tentativo di modifica fallirà e verrà mostrato un dialog esplicativo dell'errore.

Quando l'utente preme '+', accede al dialog di **Track Creation** dove può inserire tutti i metadati necessari alla creazione della traccia. Anche in questo caso, qualora l'utente inserisca dei valori non coerenti alle regole di business, la creazione fallirà e verrà contestualmente mostrato un dialog di errore.

Infine, dalla schermata All Tracks, premendo l'icona del cestino '🗑️', è possibile eliminare una traccia. Nello specifico, quando l'utente interagisce con il pulsante in questione, si attiva un dialog di **Remove Track**, che permette di confermare l'operazione di cancellazione oppure di annullarla.

Flusso 3: Homepage, Playlist Details e All Tracks

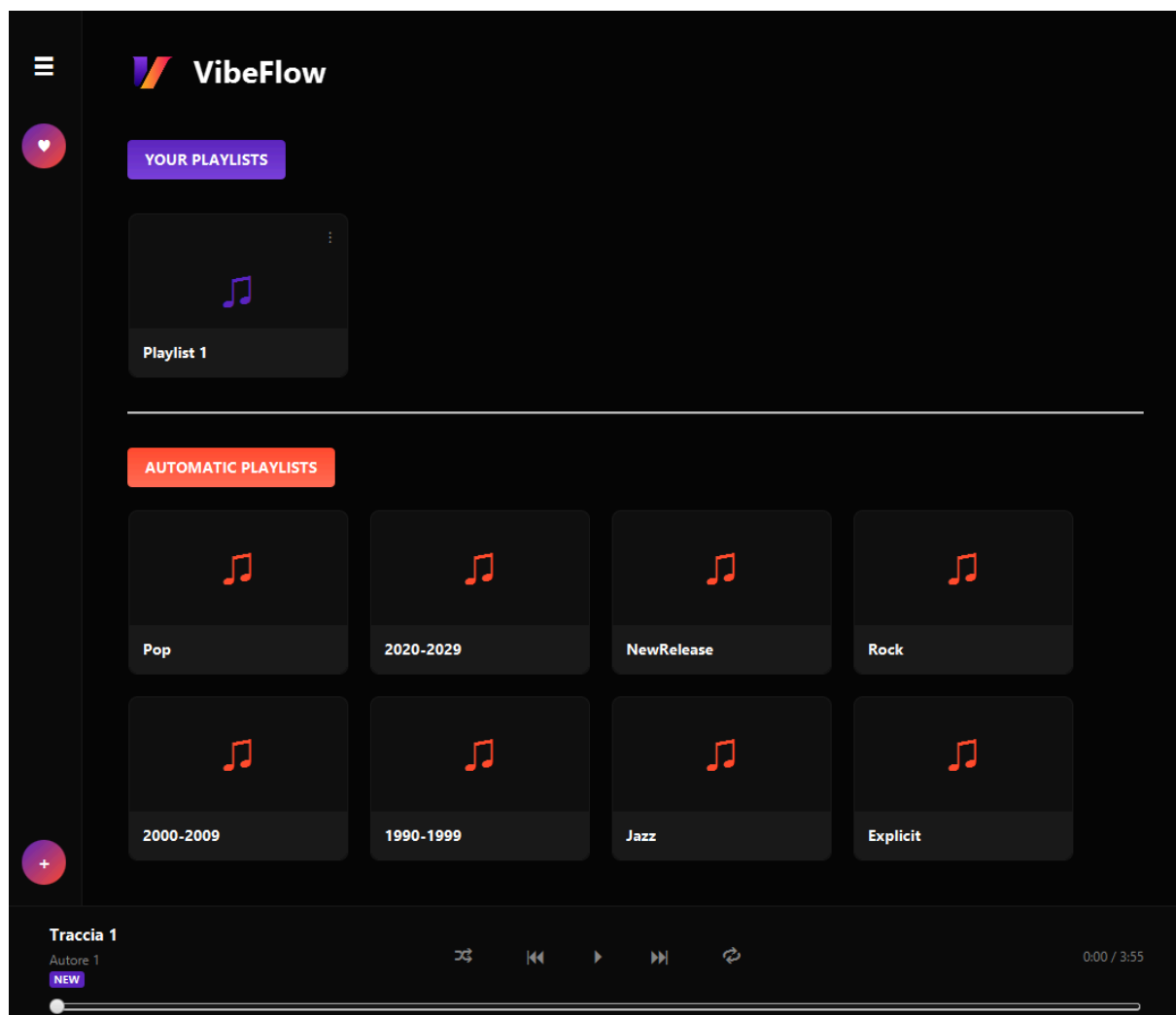


Dalla **Homepage**, selezionando una playlist dalla sezione “*Your Playlists*”, l’utente accede alla schermata **Playlist Details**, dove visualizza i brani contenuti e ne può controllare la riproduzione. Da qui, premendo il ‘🗑️’ su un brano, viene mostrato un *dialog* di conferma contestuale con il nome della traccia e della playlist di cui fa parte. Invece, premendo il pulsante ‘+’, l’utente viene reindirizzato alla schermata **All Tracks**, dove può selezionare quali brani aggiungere alla playlist corrente.

4.2 VibeFlow GUI

La presente sezione descrive l'interfaccia grafica dell'applicazione **VibeFlow** nella sua versione definitiva. L'interfaccia adotta un tema visivo scuro, elementi testuali bianchi e accenti cromatici in viola e arancione, garantendo un'esperienza visiva coerente e riconoscibile in tutte le schermate. Ciascuna interfaccia viene analizzata nel dettaglio, evidenziandone i **componenti fondamentali** e le **opzioni operative** messe a disposizione dell'utente.

Homepage



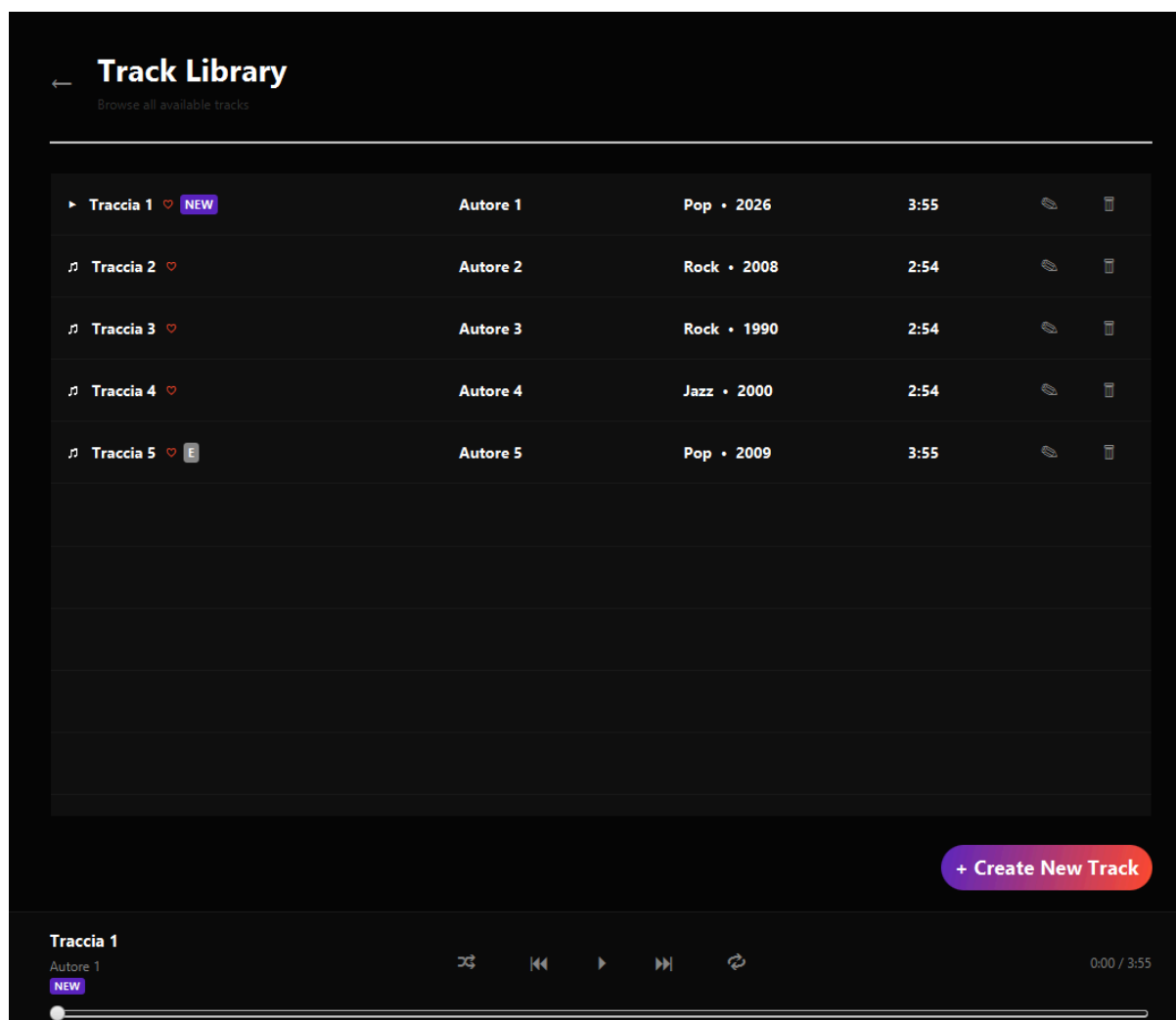
Homepage di VibeFlow

La Homepage costituisce il punto di accesso principale all'applicazione. In alto a sinistra sono presenti il logo e il nome **VibeFlow**. La schermata è suddivisa verticalmente in due sezioni principali, separate da una linea divisoria orizzontale, e presenta in basso la barra del Media Player persistente.

- **Icona Hamburger (≡):** posizionata in alto a sinistra, naviga direttamente alla schermata Track Library.

- **Icona Preferiti (♥):** pulsante circolare con icona cuore che permette di accedere alla playlist contenente tutti i brani contrassegnati come preferiti.
- **Pulsante '+':** pulsante circolare posizionato in basso a sinistra, apre il dialog di creazione di una nuova playlist.
- **Sezione “Your Playlists”:** etichettata con un badge viola, mostra le playlist create manualmente dall'utente sotto forma di card. Ogni card presenta un'icona musicale viola, il nome della playlist e un'icona a tre punti verticali (⋮) che apre un menu contestuale con le opzioni di modifica ed eliminazione.
- **Sezione “Automatic Playlists”:** etichettata con un badge arancione, mostra le raccolte generate automaticamente dal sistema. Le playlist automatiche sono generate in base al genere musicale, al decennio di pubblicazione, alla presenza di contenuto esplicito, alla pubblicazione nell'anno corrente e al numero di riproduzioni. Ogni card è rappresentata da un'icona musicale arancione e il nome della categoria.
- **Media Player:** barra fissa nella parte inferiore della finestra. Mostra la label “No track playing” quando nessun brano è in riproduzione, oppure il titolo e l'autore del brano corrente. Contiene i controlli di flusso (Shuffle, Skip Backward, Play/Pause, Skip Forward, Loop), la barra di avanzamento e il contatore temporale (tempo trascorso/durata totale). Il comportamento dettagliato dei controlli è descritto nella sezione Playlist Details.

Track Library



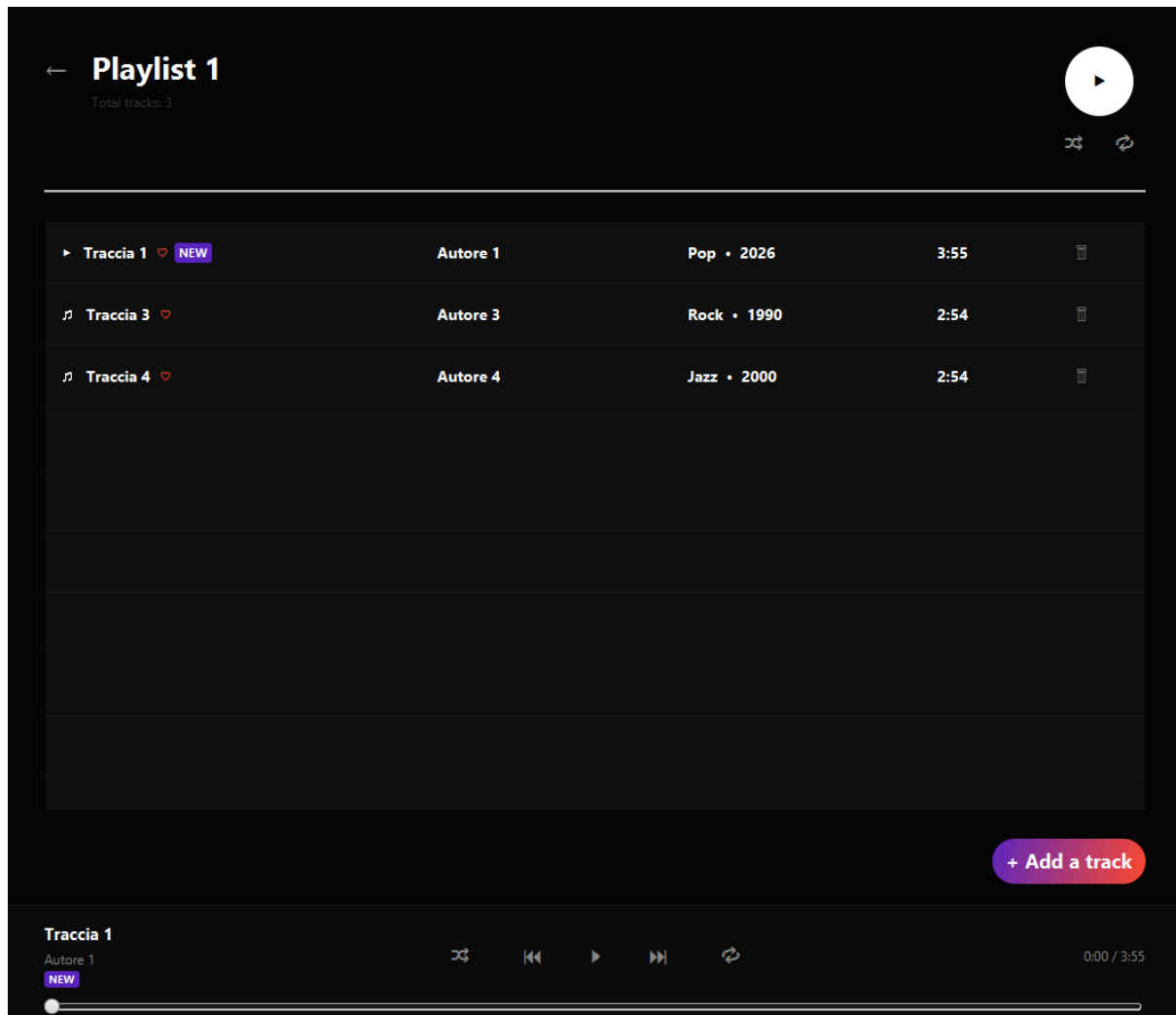
Schermata Track Library

La schermata **Track Library** è accessibile dall'Homepage tramite l'icona Hamburger (≡) e mostra la libreria globale di tutti i brani presenti nel sistema.

- **Intestazione:** titolo "*Track Library*" con sottotitolo "*Browse all available tracks*". La freccia indietro (←) consente di tornare alla Homepage.
- **Lista Tracce:** elenco verticale di tutti i brani del sistema. Ogni riga contiene:
 - **Indicatore di stato:** icona Play (▶) se in riproduzione, icona musicale (🎵) altrimenti.
 - **Titolo del brano** e le eventuali icone relative ai tag: '♥' per i Preferiti, 'E' per Explicit, 'NEW' per New Release.
 - **Autore, Genere • Anno, Durata.**
 - **Icona** '🔧': apre il dialog *Edit Track* per la modifica dei metadati del brano.

- **Icona ‘🗑️’**: elimina definitivamente il brano dal sistema, previa conferma tramite dialog.
- **Pulsante “+ Create New Track”**: pulsante viola posizionato in basso a destra, apre il dialog di creazione di un nuovo brano.
- **Media Player Persistente**: barra fissa nella parte inferiore per la riproduzione dei brani.

Playlist Details



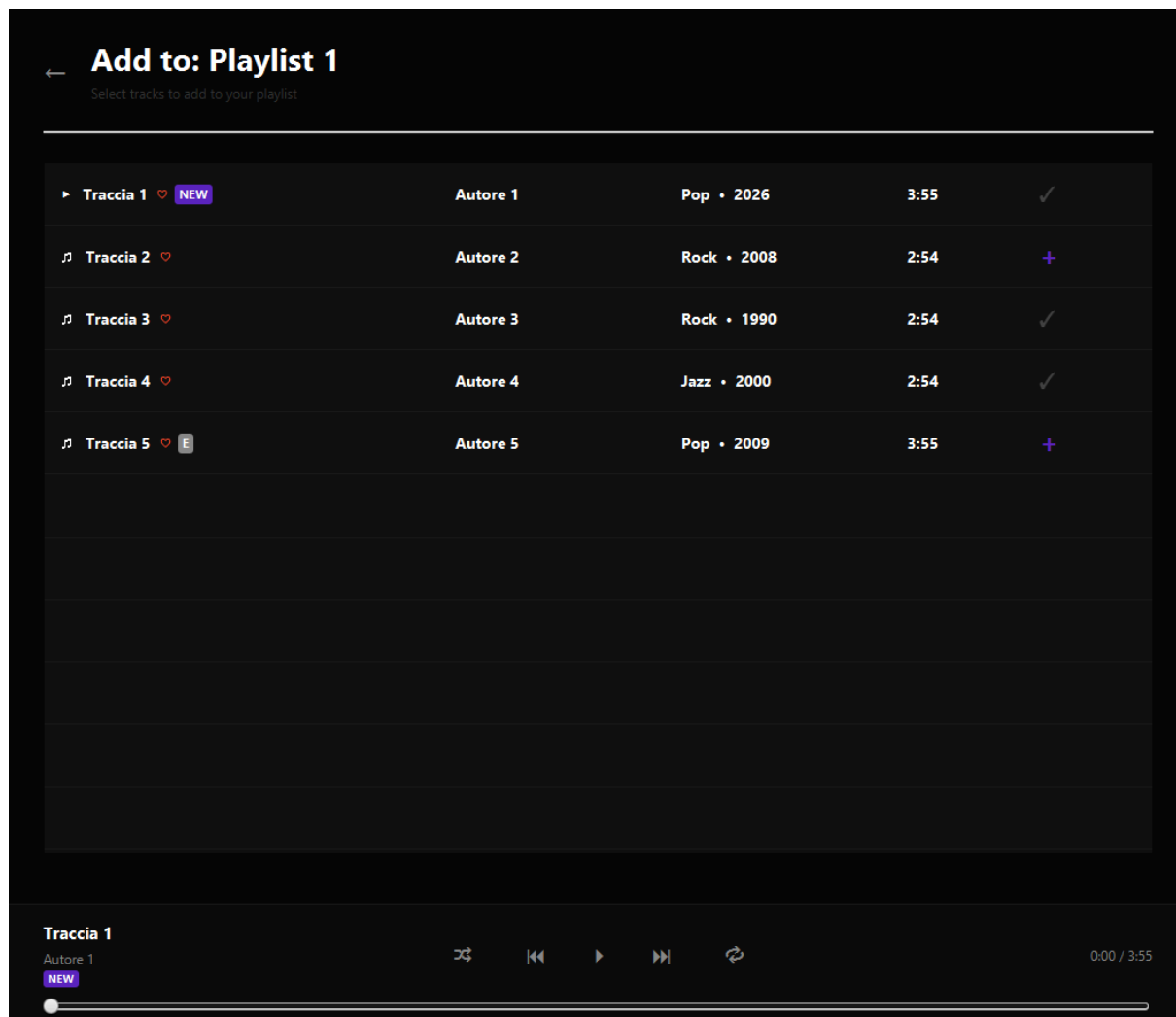
Schermata Playlist Details

La schermata **Playlist Details** mostra il contenuto di una playlist selezionata, consente la gestione dei brani e il controllo della riproduzione.

- **Intestazione**: mostra in alto a sinistra il nome della playlist in evidenza e il contatore del numero totale di tracce. Una freccia indietro (←) consente di tornare alla Homepage.
- **Pannello di Controllo Master**: posizionato in alto a destra, contiene:
 - **Pulsante Play/Pause**: pulsante circolare bianco che avvia o mette in pausa la riproduzione dell'intera playlist.

- **Icona Shuffle:** attiva o disattiva la modalità di riproduzione casuale.
- **Icona Loop:** attiva o disattiva la modalità di ripetizione della playlist.
- **Lista dei Brani:** elenco verticale delle tracce incluse nella playlist. Ogni riga contiene:
 - **Indicatore di stato:** icona Play (▶) se il brano è in riproduzione, icona musicale (🎵) altrimenti.
 - **Titolo del brano** e le eventuali icone relative ai tag: ♥ per i Preferiti, E per Explicit, **NEW** per New Release.
 - **Autore, Genere • Anno, Durata:** informazioni del brano disposte orizzontalmente.
 - **Icona** '🗑️' che permette di rimuovere il brano dalla playlist (non dalla libreria globale), previa conferma tramite dialog.
- **Pulsante '+ Add a track':** pulsante viola posizionato in basso a destra, reindirizza alla schermata *Add to Playlist* per aggiungere nuovi brani alla raccolta corrente.
- **Media Player Persistente:** barra fissa nella parte inferiore per la riproduzione dei brani.

Add to Playlist

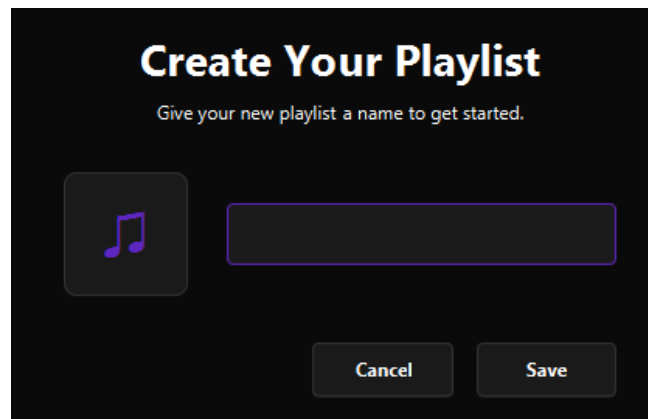


Schermata Add to Playlist

Questa schermata è una variante contestuale della Track Library, attivata premendo il pulsante “+ Add a track” dalla schermata Playlist Details. Il suo scopo è consentire all’utente di selezionare quali brani aggiungere alla playlist corrente.

- **Intestazione:** mostra il titolo “Add to: [Nome Playlist]” e il sottotitolo “Select tracks to add to your playlist”. La freccia indietro (←) consente di tornare alla schermata Playlist Details.
- **Lista Tracce:** elenco di tutti i brani disponibili nella libreria globale. Ogni riga espone titolo ed eventuali tag applicati, autore, genere • anno e durata.
- **Indicatore di Appartenenza:** a destra di ogni riga è presente un’icona che indica lo stato del brano rispetto alla playlist corrente: il simbolo ‘✓’ segnala che il brano è già incluso nella playlist; il simbolo ‘+’ indica che il brano può essere aggiunto.
- **Media Player Persistente:** barra fissa nella parte inferiore, identica a quella delle altre schermate.

Playlist Creation

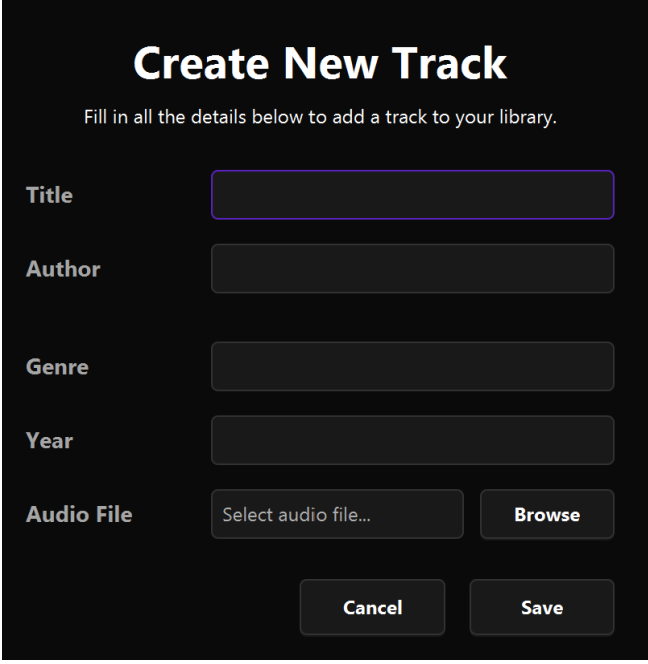


Dialog Create Your Playlist

L'attivazione del processo di creazione di una nuova playlist avviene mediante l'interazione con il tasto preposto nella schermata principale. Tale azione determina la visualizzazione di una finestra modale, il cui stile risulta in linea con l'estetica complessiva dell'applicazione, articolata nei componenti descritti di seguito:

- **Titolo “Create Your Playlist”:** intestazione del dialog, accompagnata dal sottotitolo “*Give your new playlist a name to get started.*”
- **Icona grafica:** area quadrata scura con icona musicale viola, rappresentativa della copertina della playlist.
- **Campo di Input Name:** campo di testo con bordo viola attivo quando selezionato, dedicato all'inserimento del nome della nuova playlist. Il sistema impedisce il salvataggio se il campo è vuoto o se il nome è già esistente, mostrando in entrambi i casi un dialog di errore coerente.
- **Pulsante “Cancel”:** chiude il dialog senza apportare modifiche.
- **Pulsante “Save”:** conferma la creazione della playlist dopo la validazione del campo.

Track Creation

A dark-themed modal dialog box titled "Create New Track". Below the title is a subtitle: "Fill in all the details below to add a track to your library." The dialog contains five input fields: "Title", "Author", "Genre", and "Year", each with a dark gray rectangular input area. The "Audio File" field consists of a text input with the placeholder "Select audio file..." and a "Browse" button to its right. At the bottom of the dialog are two buttons: "Cancel" and "Save".

Create New Track

Fill in all the details below to add a track to your library.

Title

Author

Genre

Year

Audio File **Browse**

Cancel **Save**

Dialog Create New Track

Il dialog **Create New Track** si apre come finestra modale. Raccoglie tutti i metadati necessari all'inserimento di un nuovo brano nella libreria globale.

- **Titolo "Create New Track"**: intestazione del dialog, con sottotitolo *"Fill in all the details below to add a track to your library."*
- **Campi obbligatori: Title, Author, Genre, Year. Audio File**. Il campo attivo è evidenziato con bordo viola.
- **Campo Audio File**: campo di testo con placeholder *"Select audio file..."* e pulsante **Browse** per la selezione del file audio con estensione *' .mp3 '* dal FileSystem.
- **Pulsante "Cancel"**: chiude il dialog senza salvare.
- **Pulsante "Save"**: avvia la validazione e, in caso di successo, aggiunge il brano alla libreria. In caso di errore viene mostrato un dialog di notifica.

Edit Track

Edit Track

Update the details of the selected track.

Title Traccia 5

Author Autore 5

Duration (s) 235

Genre Pop

Year 2009

File C:\Users\emanu\Downloads\Dr. L ...

Cancel Save

Dialog Edit Track

Il dialog **Edit Track** è accessibile dalla Track Library tramite l'icona '📝' presente su ciascuna riga. Si presenta precompilato con i metadati correnti del brano selezionato, consentendo all'utente di aggiornarne le informazioni.

- **Titolo “Edit Track”**: intestazione del dialog, con sottotitolo “Update the details of the selected track.”
- **Campi editabili: Title, Author, Duration (s), Genre, Year, Audio File**. Tutti i campi sono precompilati con i valori correnti del brano. Il campo attivo è evidenziato con bordo viola.
- **Campo File**: mostra il percorso del file audio attualmente associato al brano. Il pulsante ... consente di selezionare un file sostitutivo dal FileSystem.
- **Pulsante “Cancel”**: chiude il dialog senza salvare le modifiche.
- **Pulsante “Save”**: conferma le modifiche apportate dopo la validazione dei campi. In caso di errore viene mostrato un dialog di notifica.

4.4 Funzionalità del Sistema

La presente sezione descrive le funzionalità implementate nell'applicazione VibeFlow. Le funzionalità sono organizzate per area tematica: gestione della libreria musicale, gestione delle playlist, generazione automatica delle playlist, riproduzione audio e sistema di tag.

Gestione della Libreria Musicale

L'utente dispone di una libreria personale in cui sono raccolti tutti i brani caricati nell'applicazione. Per ogni brano è possibile:

1. **Creare e aggiungere un nuovo brano:** l'utente compila i campi titolo, autore, genere e anno di pubblicazione, e seleziona il file audio. La durata del brano viene rilevata automaticamente dal file. Tutti i campi sono obbligatori; in caso di dati mancanti o non validi, il sistema segnala l'errore con un messaggio.
2. **Modificare un brano esistente:** l'utente può aggiornare in qualsiasi momento i dati di un brano già presente in libreria, incluso il file audio associato. Le modifiche vengono riflesse globalmente in tutte le playlist che contengono quel brano.
3. **Eliminare un brano:** il brano viene rimosso dalla libreria e da tutte le playlist in cui era presente. Prima dell'eliminazione, il sistema chiede conferma all'utente.
4. **Contrassegnare un brano come Preferito:** l'utente può aggiungere o rimuovere il tag Preferito da qualsiasi brano. L'aggiornamento è immediato e visibile tramite l'icona cuore.
5. **Esecuzione dei brani:** attraverso un doppio clic su una traccia, l'utente può avviarne la riproduzione immediata. Le funzioni di controllo permettono inoltre di navigare tra i contenuti della Libreria Musicale sfruttando le opzioni di riproduzione Sequenziale, Shuffle o Loop.

Gestione delle Playlist

L'utente può creare e gestire le proprie playlist personalizzate. Le operazioni disponibili sono:

1. **Creare una nuova playlist:** l'utente assegna un nome alla playlist. Il sistema verifica che il nome non sia già utilizzato da un'altra playlist e che non sia vuoto.
2. **Rinominare una playlist:** è possibile modificare il nome di una playlist in qualsiasi momento, con le stesse verifiche applicate in fase di creazione.
3. **Eliminare una playlist:** la playlist viene rimossa dall'applicazione. I brani contenuti al suo interno non vengono eliminati dalla libreria.
4. **Aggiungere brani a una playlist:** dalla vista della playlist, l'utente può sfogliare la libreria globale e aggiungere i brani desiderati. Il sistema impedisce di inserire lo stesso brano più di una volta nella stessa playlist.

5. **Rimuovere un brano da una playlist:** il brano viene rimosso dalla playlist selezionata, senza essere eliminato dalla libreria. Prima della rimozione il sistema mostra un dialog di conferma con il nome della traccia e della playlist coinvolte.
6. **Contrassegnare un brano come Preferito:** l'utente può aggiungere o rimuovere il tag Preferito da qualsiasi brano. L'aggiornamento è immediato e visibile tramite l'icona cuore.
7. **Esecuzione dei brani:** attraverso un doppio clic su una traccia, l'utente può avviarne la riproduzione immediata. Alternativamente, è possibile riprodurre la playlist dal pulsante dedicato. Le funzioni di controllo permettono inoltre di navigare tra i contenuti della playlist sfruttando le opzioni di riproduzione Sequenziale, Shuffle o Loop.
8. **Riordinare i brani:** l'utente può modificare l'ordine dei brani all'interno di una playlist.

Creazione di Playlist Automatiche

Oltre alle playlist create manualmente, il sistema genera automaticamente una serie di raccolte basate sul contenuto della libreria. Queste playlist si aggiornano in tempo reale ogni volta che la libreria viene modificata e non possono essere rinominate o eliminate dall'utente. Vibeflow consente la creazione di quattro tipi di collezioni automatiche: playlist per **genere**, per **decennio**, per **Tag** e per **brani più ascoltati**.

Queste playlist automatiche riflettono i cambiamenti globali della libreria, per cui se tutti i brani di un genere, con un certo tag oppure di un certo decennio vengono eliminati, la relativa playlist automatica scompare di conseguenza.

1. **Playlist per Genere:** per ogni genere musicale presente in libreria viene creata automaticamente una playlist contenente tutti i brani di quel genere.
2. **Playlist per Decennio:** i brani vengono raggruppati automaticamente per decennio di pubblicazione.
3. **Playlist Preferiti:** raccoglie i brani contrassegnati come preferiti (♥).
4. **Playlist Explicit:** raccoglie automaticamente tutti i brani contrassegnati come contenuto esplicito. Il tag Explicit viene assegnato dal sistema al momento dell'inserimento del brano e non è modificabile dall'utente.
5. **Playlist New Release:** raccoglie automaticamente tutti i brani pubblicati nell'anno corrente. Il sistema assegna il badge 'NEW' a questi brani in fase di inserimento; se l'anno di pubblicazione non coincide con quello corrente, il badge non viene assegnato.
6. **Playlist Most Played:** raccoglie i brani più ascoltati, ordinati dal brano riprodotto più volte a quello con meno ascoltato. Si aggiorna automaticamente al termine della riproduzione di un brano.

Riproduzione Musicale

L'utente può riprodurre musica in qualsiasi momento, sia da una playlist a sua scelta, sia dalla libreria delle tracce.

Il player musicale è sempre visibile nella parte inferiore dello schermo e rimane attivo durante tutta la navigazione nell'applicazione. I controlli disponibili sono:

1. **Play/Pausa:** avvia o mette in pausa la riproduzione del brano corrente.
2. **Brano successivo:** passa al brano successivo nella coda di riproduzione.
3. **Brano precedente:** il pulsante ha una doppia funzione: premuto entro i primi 10 secondi del brano, consente di tornare al brano precedente; premuto oltre tale soglia, riavvia il brano corrente dall'inizio.
4. **Barra di avanzamento:** mostra l'avanzamento temporale del brano in riproduzione, consentendo all'utente di riprodurre un punto specifico della traccia spostando il cursore sullo slider.
5. **Modalità sequenziale:** modalità di default in cui le tracce vengono riprodotte nell'ordine in cui sono state disposte all'interno della libreria.
6. **Modalità Shuffle:** attiva la riproduzione in ordine casuale. L'ordine originale della playlist non viene modificato.
7. **Modalità Loop:** attiva la riproduzione ciclica. Al termine dell'ultimo brano, la riproduzione riparte automaticamente dal primo, mantenendo l'ordine di default.

Annullamento delle Operazioni

L'applicazione consente di annullare le operazioni eseguite: nello specifico, VibeFlow mantiene uno storico delle ultime cinque operazioni effettuate.

Le azioni annullabili comprendono:

1. L'**aggiunta** e l'**eliminazione** di un brano dalla libreria.
2. L'**aggiunta** e la **rimozione** di un brano da una playlist.
3. La **modifica** dei metadati di un brano.
4. La **creazione** e la **rinomina** di una playlist.
5. Il **riordinamento dei brani**.